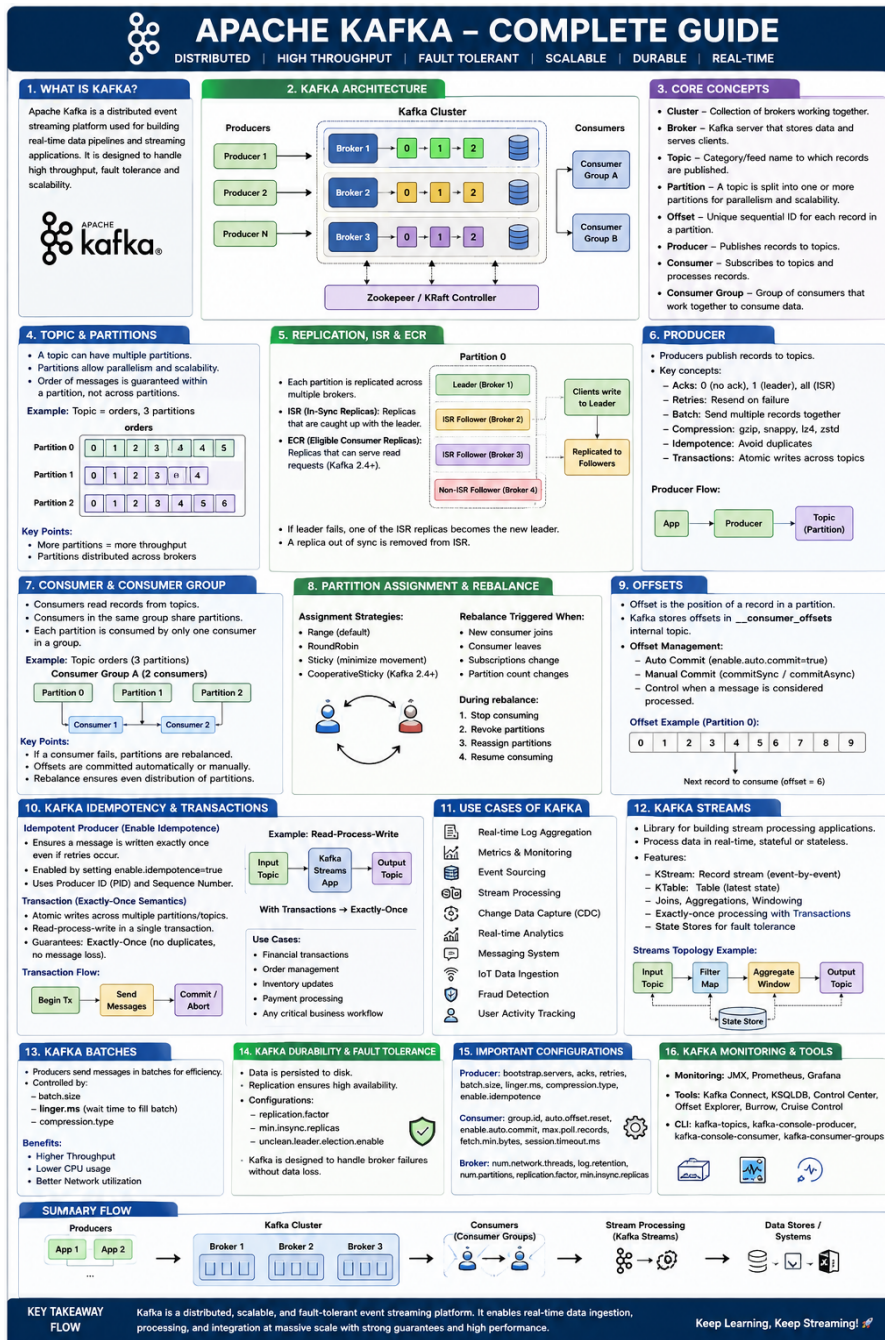


APACHE KAFKA

Complete & Advanced Guide

Architecture • Brokers • Topics • Partitions • Replication • ISR • ELR • Producers • Consumers • Offsets • Rebalancing • Idempotency • Transactions • Streams • Connect • Batching • Storage • Monitoring • Use Cases



A practical reference for backend developers, system-design preparation, interviews, and production Kafka architecture.

Contents

1. Kafka fundamentals and architecture
2. Topics, partitions and ordering
3. Replication, leader, ISR and ELR
4. Producer internals, acknowledgements and batching
5. Idempotency and transactions
6. Consumers and consumer groups
7. Partition assignment and rebalancing
8. Offsets and delivery semantics
9. Storage, retention and compaction
10. KRaft metadata architecture
11. Kafka Streams
12. Kafka Connect
13. Performance and configuration
14. Monitoring and consumer lag
15. Security concepts
16. Schema and serialization
17. Failure scenarios
18. Common Kafka use cases
19. System-design patterns
20. Interview mental model and production checklist

1. What is Apache Kafka?

Apache Kafka is a distributed event-streaming platform. Applications publish immutable records to topics; Kafka stores those records durably and allows one or many independent consumers to read and replay them.

- High-throughput publish/subscribe messaging
- Durable append-only storage
- Horizontal scaling through partitions
- Fault tolerance through replication
- Real-time stream processing
- Loose coupling between producers and consumers

Example / mental model

```
Producer -> Kafka Cluster -> Consumer
-> Consumer
-> Stream Processor
```

Key point: Kafka is best understood as a distributed, replicated event log - not merely a queue.

2. Kafka Cluster and Broker

A Kafka cluster is a collection of brokers. A broker is a Kafka server that accepts client requests and stores topic partitions.

- A cluster can contain many brokers.
- Each broker can host many partitions from many topics.
- Partition leaders are distributed across brokers to spread load.
- Clients use metadata to discover the broker currently responsible for a partition.

Example / mental model

```
Cluster
Broker 1: orders-P0, payments-P1
Broker 2: orders-P1, payments-P0
Broker 3: orders-P2, payments-P2
```

Key point: Adding brokers increases available storage and processing capacity, but partitions are the fundamental unit of parallelism.

3. Topic

A topic is a named logical stream of related events, such as orders, payments, inventory-events, or customer-updates.

- Producers write records to topics.
- Consumers subscribe to topics.
- A topic is split into one or more partitions.
- Topics can have independent retention, cleanup and replication settings.

Example / mental model

```
orders -> P0 | P1 | P2
```

Key point: A topic is a logical namespace; partitions are where ordering, storage and parallelism actually happen.

4. Partition

A partition is an ordered, append-only sequence of records. Each new record receives an offset within that partition.

- Ordering is guaranteed within one partition.
- Different partitions can be processed concurrently.
- More partitions generally allow greater producer and consumer parallelism.
- A partition has exactly one active leader at a time.

Example / mental model

```
P0: [0:A] [1:B] [2:C] [3:D]
P1: [0:X] [1:Y] [2:Z]
```

Key point: Kafka does not provide a single global ordering across all partitions of a topic.

5. Partitioning and Message Keys

The producer determines the destination partition. When a key is supplied, a partitioner normally maps the same key consistently to the same partition while the partition count remains unchanged.

- Use keys when events for the same business entity must preserve order.
- Examples: customerId, orderId, accountId, deviceId.
- Poor key distribution can create hot partitions.
- Changing the number of partitions can change key-to-partition mapping.

Example / mental model

```
key=customer-101 -> hash -> P2
key=customer-101 -> hash -> P2
key=customer-202 -> hash -> P0
```

Key point: Choose partition keys based on both ordering requirements and load distribution.

6. Replication

Kafka replicates partitions across brokers so data remains available when a broker fails.

- replication.factor=3 means three replicas of each partition exist.
- One replica is leader; others follow it.
- Followers fetch records from the leader.
- Replica placement should span failure domains where possible.

Example / mental model

```
P0 Leader: Broker 1
P0 Follower: Broker 2
P0 Follower: Broker 3
```

Key point: Replication provides fault tolerance; it does not increase consumer parallelism for a partition.

7. Leader and Followers

All normal reads and writes for a partition are coordinated through its leader. Followers replicate the leader's log.

- Producer discovers the leader through metadata.
- Leader appends accepted records.
- Followers continuously fetch and replicate.
- If the leader fails, Kafka elects an eligible replacement.

Example / mental model

Producer -> P0 Leader (B1) -> replication -> B2, B3

Key point: Leadership can move between replicas during failures, maintenance and balancing.

8. ISR - In-Sync Replicas

ISR is the set of replicas considered sufficiently synchronized with the leader. ISR is central to Kafka's durability and leader-election behavior.

- The leader itself is normally in ISR.
- Followers that fall too far behind can leave ISR.
- Caught-up replicas can rejoin ISR.
- With acks=all, durability is closely related to ISR and min.insync.replicas.

Example / mental model

ISR = {B1 leader, B2 follower}
B3 follower -> lagging -> outside ISR

Key point: ISR should be monitored closely; shrinking ISR often indicates broker, disk or network problems.

9. ELR - Eligible Leader Replicas

Eligible Leader Replicas (ELR) is a modern Kafka mechanism related to maintaining safe leader-election options when replicas leave ISR. It is distinct from ISR.

- ELR is an advanced availability/durability concept.
- It should not be confused with ordinary follower replicas.
- Cluster/version configuration determines its exact operational behavior.

Key point: For modern Kafka material, use the term ELR (Eligible Leader Replicas), not ECR.

10. Producer Write Path

A producer serializes a record, chooses a partition, accumulates records into batches and sends produce requests to the partition leader.

- Metadata lookup -> serialization -> partition selection -> batching -> network send -> acknowledgement.
- Retries handle transient failures.
- Compression is usually applied to batches.
- Producer throughput is heavily influenced by batching.

Example / mental model

Application -> KafkaProducer -> Batch -> Broker/Partition Leader

Key point: Producer tuning is a latency-versus-throughput trade-off.

11. Producer Acknowledgements

The acks setting determines how much acknowledgement the producer requires before a write is considered successful.

- acks=0: producer does not wait for acknowledgement.
- acks=1: leader acknowledges after its local write.
- acks=all: strongest normal acknowledgement mode; works with ISR and min.insync.replicas.
- For critical data, acks=all plus appropriate replication and min ISR is a common design.

Example / mental model

```
replication.factor=3
min.insync.replicas=2
acks=all
```

Key point: No single setting guarantees durability by itself; broker, topic and producer settings work together.

12. Retries and Delivery Timeout

Transient broker or network failures can cause producer requests to be retried.

- retries controls retry behavior in older tuning models.
- delivery.timeout.ms bounds total delivery time.
- Retry behavior must be considered with ordering and idempotency.
- Backpressure can appear when brokers cannot accept data fast enough.

Key point: Idempotent production is critical when retries must not create duplicate Kafka records.

13. Producer Batching

Kafka producers group records into batches to reduce request overhead and improve throughput.

- batch.size limits the target batch size.
- linger.ms allows a short wait so more records can join a batch.
- compression.type can reduce network and disk usage.
- Larger batches usually improve throughput but may add latency.

Example / mental model

```
Records -> [batch] -> ProduceRequest -> Broker
```

Key point: Batching is one of the major reasons Kafka can achieve very high throughput.

14. Compression

Kafka supports batch compression. Common codecs include gzip, snappy, lz4 and zstd.

- Compression reduces network bandwidth and storage.
- Better compression often costs more CPU.
- Compressing a batch is more efficient than compressing tiny records independently.

Key point: Select compression based on CPU, bandwidth, message size and latency requirements.

15. Idempotent Producer

Idempotent production prevents duplicates caused by producer retries for a partition by using producer identity and sequence numbers.

- A retry of the same produce operation can be recognized.
- This protects against a major source of duplicate records.
- It does not make arbitrary external side effects idempotent.

Example / mental model

```
send #42 -> broker stores  
ack lost -> retry #42  
broker recognizes retry -> no duplicate append
```

Key point: Idempotency solves producer retry duplication; business-level deduplication may still be needed elsewhere.

16. Kafka Transactions

Kafka transactions allow writes to multiple partitions/topics to be committed or aborted atomically. They can also coordinate consumed offsets with produced records.

- Begin transaction.
- Produce one or more records.
- Optionally send consumed offsets to the transaction.
- Commit or abort.
- Consumers using `read_committed` avoid exposing aborted transactional records.

Example / mental model

```
Begin Tx -> write A -> write B -> offsets -> COMMIT  
\-> ABORT on failure
```

Key point: Transactions are especially useful for Kafka consume-process-produce pipelines.

17. Delivery Semantics

Kafka applications are commonly discussed in terms of at-most-once, at-least-once and exactly-once processing semantics.

- At-most-once: loss is possible, duplicate processing is minimized.
- At-least-once: records can be retried/reprocessed, so duplicates are possible.
- Exactly-once: Kafka transactions/Streams can make Kafka state and output effects atomic within the supported processing model.

Key point: Exactly-once inside Kafka does not automatically make a database, REST API or email side effect exactly-once.

18. Consumer

A Kafka consumer polls brokers for records. Pull-based consumption gives applications control over processing rate and batching.

- Consumers deserialize records.
- poll() fetches available data.
- Consumers track positions using offsets.
- Applications decide when processing is complete and when offsets should be committed.

Example / mental model

```
Broker -> poll() -> Consumer -> Business Logic
```

Key point: Consumer design must coordinate polling, processing time, failures and offset commits.

19. Consumer Group

A consumer group is a set of consumers cooperating to process a topic subscription.

- Partitions are distributed among group members.
- In the traditional group model, one partition is assigned to at most one consumer in a group at a time.
- Different consumer groups can independently read the same topic.
- If consumers exceed partitions, some consumers may be idle.

Example / mental model

```
Topic: P0 P1 P2
Group A: C1->P0,P1 | C2->P2
Group B: C3->P0 | C4->P1 | C5->P2
```

Key point: Partitions determine the maximum useful parallelism of a traditional consumer group.

20. Partition Assignment Strategies

Kafka consumer groups use assignment strategies to decide which member owns which partitions.

- Range: groups ranges of partitions per topic.
- RoundRobin: distributes partitions across consumers.
- Sticky: attempts balanced assignment while minimizing movement.
- CooperativeSticky: supports incremental cooperative rebalancing.

Key point: Assignment strategy affects rebalance disruption and partition movement.

21. Rebalancing

A rebalance changes partition ownership among consumers in a group.

- Triggered by membership or subscription changes and relevant metadata changes.
- Consumers may revoke and acquire partitions.
- Frequent rebalances reduce throughput and can cause duplicate work if processing/commits are poorly coordinated.
- Cooperative rebalancing can reduce stop-the-world behavior.

Example / mental model

```
C1:P0,P1 + C2:P2
C3 joins
=> C1:P0 | C2:P1 | C3:P2
```

Key point: Stable membership and correct timeout settings are important for production consumers.

22. Offsets

An offset is a monotonically increasing position of a record within a partition.

- Offsets are partition-local.
- A consumer has a current position and can commit progress.
- Offsets enable restart, replay and independent consumption.
- Seeking to an earlier offset enables reprocessing.

Example / mental model

```
P0: 0 1 2 3 4 5 6
^ committed=3
next processing position may be 4
```

Key point: Offsets identify position, not globally unique message identity.

23. Offset Commit and `__consumer_offsets`

Consumer groups persist committed offsets in Kafka's internal `__consumer_offsets` topic.

- Auto commit trades simplicity for less explicit processing control.
- Manual synchronous commits are simple but blocking.
- Manual asynchronous commits reduce blocking but need careful error handling.
- Commit timing affects duplicate-versus-loss behavior.

Example / mental model

```
poll -> process -> commit offset
```

Key point: Commit after successful processing for common at-least-once designs.

24. `auto.offset.reset`

When a consumer group has no valid committed offset, `auto.offset.reset` determines where it starts.

- `earliest`: begin from the earliest retained records.
- `latest`: begin from newly arriving records.
- `none`: fail when no valid offset exists, depending on client behavior/configuration.

Key point: This setting is not a normal 'always start from' switch; committed offsets take precedence when valid.

25. Poll Loop and Consumer Liveness

Kafka consumers must continue polling within configured limits so the group knows they are healthy and making progress.

- `max.poll.records` limits records returned per poll.
- `max.poll.interval.ms` limits allowed time between poll calls.
- `session.timeout.ms` and heartbeat behavior participate in failure detection.
- Long processing should be designed so it does not accidentally trigger rebalances.

Key point: Slow downstream processing is a common cause of consumer instability and lag.

26. Kafka Storage Model

Each partition is stored as an append-only log divided into segments with index files that help locate records efficiently.

- Sequential disk I/O is efficient.
- Old segments can be deleted according to retention.
- Kafka relies heavily on the operating system page cache.
- Records are immutable after append.

Example / mental model

```
partition/  
00000000000000000000.log  
00000000000000000000.index  
00000000000012345678.log
```

Key point: Kafka storage is optimized for sequential append and sequential consumption.

27. Retention

Retention controls how long or how much log data Kafka keeps when `cleanup.policy` includes `delete`.

- `retention.ms` controls time-based retention.
- `retention.bytes` can bound partition size.
- segment settings influence when old data becomes eligible for cleanup.
- Retention is independent of whether every consumer has read a record.

Key point: Kafka is not a traditional queue that deletes a record immediately after one consumer reads it.

28. Log Compaction

Compaction preserves the latest value for each key over time, making Kafka useful for reconstructing current state.

- `cleanup.policy=compact` enables compaction.
- Older values for a key become eligible for removal.
- Tombstone records can represent deletion.
- Compaction and delete retention can be combined.

Example / mental model

```
k1=A, k2=X, k1=B, k1=C  
Eventually latest state approximates: k1=C, k2=X
```

Key point: Compaction preserves key state semantics, not a perfectly minimal file at every instant.

29. KRaft Architecture

Modern Kafka uses the KRaft metadata quorum rather than ZooKeeper for cluster metadata management.

- Controllers participate in a metadata quorum.
- The active controller manages cluster metadata changes.
- Brokers receive metadata updates from the controller quorum.
- Topic, partition and broker metadata are maintained through Kafka's own consensus architecture.

Example / mental model

```
Clients -> Brokers
^
Metadata
^
KRaft Controllers
```

Key point: KRaft is the architecture to learn for current Kafka deployments.

30. Kafka Streams

Kafka Streams is a Java library for building stream-processing applications directly on Kafka.

- Supports map, filter, branch, join and aggregate operations.
- Supports stateless and stateful processing.
- Uses Kafka topics for input/output and changelog/repartition needs.
- Scales by running multiple application instances.

Example / mental model

```
Input Topic -> Filter -> Map -> Aggregate/Window -> Output Topic
```

Key point: Kafka Streams is a library embedded in your application, not a separate processing cluster.

31. KStream and KTable

KStream models an event stream where each record is an independent event. KTable models the latest value/state for each key.

- KStream: every event is meaningful.
- KTable: updates for a key replace its current logical value.
- Stream-table and table-table joins enable powerful event/state processing patterns.

Example / mental model

```
KStream: order-created, order-updated, order-cancelled
KTable: customer-101 -> ACTIVE
```

Key point: Choose the abstraction based on whether you model events or evolving state.

32. State Stores and Changelog Topics

Stateful Kafka Streams operations keep local state stores. Changelog topics can be used to restore state after failure.

- Aggregations and joins often require state.
- Local state provides fast access.
- Changelog records make state recoverable.
- Standby replicas can reduce restoration time in some designs.

Example / mental model

```
Input -> Processor <-> Local State Store
-> Changelog Topic
```

Key point: Stateful stream processing requires planning for restoration time and storage.

33. Windowing

Windowing groups events by time so infinite streams can be aggregated over bounded periods.

- Tumbling windows: fixed, non-overlapping.
- Hopping windows: fixed size with overlap.
- Sliding windows: dynamically defined around event pairs/time differences.
- Session windows: activity-based sessions separated by inactivity gaps.

Example / mental model

```
10:00-10:05 -> count  
10:05-10:10 -> count
```

Key point: Window semantics depend on event time, grace periods and late-arriving records.

34. Kafka Connect

Kafka Connect is a framework for moving data between Kafka and external systems using reusable source and sink connectors.

- Source connector: external system -> Kafka.
- Sink connector: Kafka -> external system.
- Connect workers can run standalone or distributed.
- Converters and schemas determine record representation.

Example / mental model

```
Database -> Source Connector -> Kafka -> Sink Connector -> Search/Data Lake
```

Key point: Use Connect when integration is standardizable; write custom consumers/producers when business logic dominates.

35. CDC with Kafka

Change Data Capture publishes database changes as events, often using a CDC connector such as Debezium with Kafka Connect.

- Captures inserts, updates and deletes.
- Allows multiple downstream consumers to react independently.
- Useful for cache updates, search indexing, analytics and integration.
- Requires careful schema evolution and ordering design.

Example / mental model

```
DB transaction log -> CDC connector -> Kafka -> many consumers
```

Key point: CDC turns database changes into a reusable event stream.

36. Important Producer Configurations

Producer configuration determines durability, throughput, latency and failure behavior.

- bootstrap.servers
- acks

- enable.idempotence
- batch.size
- linger.ms
- compression.type
- delivery.timeout.ms
- request.timeout.ms
- max.in.flight.requests.per.connection
- buffer.memory

Key point: Tune from measured workload requirements rather than copying one universal configuration.

37. Important Consumer Configurations

Consumer configuration determines group behavior, polling, fetch size and offset management.

- group.id
- enable.auto.commit
- auto.offset.reset
- max.poll.records
- max.poll.interval.ms
- session.timeout.ms
- heartbeat.interval.ms
- fetch.min.bytes
- fetch.max.wait.ms

Key point: Processing time and poll/liveness settings must be designed together.

38. Important Broker/Topic Configurations

Broker and topic settings control partition count, replication, durability and retention.

- num.partitions
- default.replication.factor / replication.factor
- min.insync.replicas
- unclean leader election behavior
- retention.ms
- retention.bytes
- cleanup.policy
- segment.bytes

Key point: Replication factor and min ISR are architecture decisions, not just tuning knobs.

39. Consumer Lag

Consumer lag measures how far a consumer group is behind the latest available records in a partition.

- Approximation: log end offset - consumer progress.
- High lag can mean slow processing or insufficient capacity.
- Lag should be observed per partition, not only as one total number.
- A hot partition can create lag even when other partitions are healthy.

Example / mental model

```
Latest offset=1000
Consumer position/committed progress=850
Lag≈150
```

Key point: Always investigate lag together with processing latency, rebalance events, broker health and downstream dependencies.

40. Monitoring

Production Kafka needs monitoring across brokers, controllers, producers and consumers.

- Broker request latency and throughput
- Bytes in/out
- Disk usage
- Under-replicated/offline partitions
- ISR shrink/expand
- Consumer lag
- Producer errors/retries
- Consumer poll/processing latency
- Controller health

Key point: JMX metrics are commonly exported to systems such as Prometheus and visualized in Grafana.

41. Serialization and Schema

Kafka stores bytes; producers serialize objects and consumers deserialize them. A schema strategy prevents incompatible changes.

- Common formats: JSON, Avro, Protobuf and raw bytes.
- Schema registries can manage versions and compatibility.
- Backward/forward compatibility rules should be defined before services evolve independently.

Key point: Event contracts are APIs; schema evolution deserves the same discipline as REST API evolution.

42. Kafka Security

Kafka supports authentication, authorization and encryption mechanisms for production environments.

- TLS protects traffic in transit.
- SASL mechanisms authenticate clients.
- ACLs or other authorization mechanisms control operations.
- Secrets and credentials should be managed outside application source code.

Key point: Security should be designed for brokers, clients, Connect, Streams and operational tooling.

43. Failure Scenarios

Kafka architecture becomes clearer when examined through failures.

- Broker failure: partition leaders may move to eligible replicas.
- Consumer failure: group rebalances and partitions move.
- Producer timeout: retry behavior and idempotency matter.
- Slow disk/network: ISR may shrink.
- Long consumer processing: poll interval can be exceeded and rebalance triggered.

Key point: Design Kafka around failure recovery, not only the happy path.

44. Common Use Cases

Kafka is widely used when many systems need to publish, consume, replay or process event streams independently.

- Event-driven microservices
- Real-time analytics
- Event sourcing
- CDC
- Log aggregation
- IoT ingestion
- Fraud detection
- Notifications
- Data pipelines
- Audit/event streams

Key point: Kafka is strongest when data is naturally represented as an ordered stream of events.

45. Event-Driven Microservices

Kafka decouples services by replacing synchronous chains with durable events where appropriate.

- Producer does not need every consumer to be online.
- New consumers can be added without changing the producer.
- Consumers can replay history when retained.
- Asynchronous design improves decoupling but introduces eventual consistency.

Example / mental model

```
Order Service -> order-created -> Kafka
-> Inventory Service
-> Payment Service
-> Notification Service
```

Key point: Do not replace every synchronous API with Kafka; choose messaging where asynchronous event semantics fit.

46. Ordering in Real Systems

Kafka ordering is partition-scoped. If business ordering is required, related events must be routed consistently.

- Same orderId key -> same partition -> ordered append.
- Multiple partitions mean no total topic ordering.
- Increasing partitions can alter key distribution.
- Retries and parallel application processing can still affect business-visible order if designed carelessly.

Key point: Define exactly what must be ordered: per customer, account, order, device, or globally.

47. Hot Partitions and Skew

A partition can become a bottleneck when too much traffic maps to one key or a small set of keys.

- One partition leader handles that partition's traffic.
- Adding consumers does not split one partition among multiple traditional group consumers.
- Use well-distributed keys or deliberate sharding when business semantics permit.

Key point: Partition count alone does not guarantee balanced throughput.

48. Backpressure and Capacity

Kafka can absorb bursts, but downstream systems still have finite capacity.

- Consumer lag is a natural buffer signal.
- Scale consumers only up to useful partition parallelism.
- Batch database writes when safe.
- Protect downstream services with bounded concurrency and retry policies.

Key point: Kafka decouples rates; it does not eliminate capacity planning.

49. Complete End-to-End Flow

Putting the major concepts together produces a useful mental model for troubleshooting and system design.

Example / mental model

```
Application
-> Producer
-> Serialize
-> Select Partition
-> Batch/Compress
-> Partition Leader
-> Replicate to ISR
-> Acknowledge
-> Consumer Group
-> Partition Assignment
-> poll()
-> Deserialize
-> Process
-> Commit Offset
-> DB/API/Output Topic
```

Key point: At every arrow, ask: what happens on timeout, retry, crash, duplicate, rebalance or partial failure?

50. Kafka Interview Mental Model

For interviews and architecture discussions, connect every Kafka concept to one of five concerns: scalability, ordering, durability, delivery semantics and operations.

- Scalability -> partitions + brokers + consumer groups
- Ordering -> partition key
- Durability -> replication + ISR + acks + min ISR
- Delivery semantics -> offsets + idempotency + transactions
- Operations -> lag + rebalances + disk + ISR + monitoring

Key point: If you can explain these five relationships clearly, most Kafka architecture questions become much easier.

Production Design Checklist

- Define expected throughput, message size and retention.
- Choose partition key based on ordering and distribution.
- Choose partition count with growth and consumer parallelism in mind.
- Use an appropriate replication factor across failure domains.
- Set `min.insync.replicas` and producer acks intentionally.
- Use idempotent production where retries matter.
- Use transactions only where atomic Kafka operations are required.
- Design consumer commit timing around failure semantics.
- Size `max.poll` settings around real processing time.
- Monitor consumer lag per partition.
- Alert on under-replicated/offline partitions and ISR changes.
- Plan schema evolution and compatibility.
- Test broker, consumer and network failures.
- Plan retention, compaction and disk capacity.
- Secure traffic, authentication and authorization.
- Load-test the complete pipeline including downstream dependencies.

Final Takeaway

Kafka's power comes from combining partitioned logs, replication, consumer groups, offsets, batching, idempotency, transactions and stream processing. Learn each concept separately, then reason about how they interact during failures. That is the foundation for designing reliable Kafka systems.